

LOGISTIC REGRESSION AND TRUST-REGION OPTIMIZATION

PEDRO LLERENAS

ABSTRACT. We study the optimization problem arising in logistic regression for binary classification using the [MNIST dataset](#). Using this dataset, we compare the performance of steepest descent with backtracking and the dogleg trust-region method.

1. INTRODUCTION

Let $\{(x_i, y_i)\}_{i=1}^n$ be a dataset with $x_i \in \mathbb{R}^{784}$ and $y_i \in \{0, 1\}$. We define the parameter vector

$$z = \begin{pmatrix} \beta \\ \beta_0 \end{pmatrix} \in \mathbb{R}^{784+1}. \quad (1)$$

Let $\tilde{x}_i = \begin{pmatrix} x_i \\ 1 \end{pmatrix}$. The logistic model defines the probability

$$\pi_i(z) = \frac{1}{1 + \exp(-x_i^\top \beta - \beta_0)} = \frac{1}{1 + \exp(-\tilde{x}_i^\top z)}. \quad (2)$$

The log-likelihood function is

$$h(z) = \sum_{i=1}^n y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i). \quad (3)$$

Estimating the parameters corresponds to maximizing $h(z)$. For convenience, we define the negative log-likelihood

$$f(z) = -h(z), \quad (4)$$

so that the problem becomes a minimization problem.

We note that

$$\begin{aligned} \pi_i(z) &= \frac{1}{1 + \exp(-\tilde{x}_i^\top z)} \\ &= 1 - \frac{\exp(-\tilde{x}_i^\top z)}{1 + \exp(-\tilde{x}_i^\top z)} \\ &= 1 - \pi_i(-z). \end{aligned} \quad (5)$$

Therefore,

$$\begin{aligned} \nabla_z \pi_i(z) &= \pi_i^2(z) (-\exp(\tilde{x}_i^\top z) \tilde{x}_i) \\ &= \pi_i(z)(1 - \pi_i(-z)) (-\exp(-\tilde{x}_i^\top z) \tilde{x}_i) \\ &= \pi_i(z)(1 - \pi_i(z)) \tilde{x}_i. \end{aligned} \quad (6)$$

Now, we may see that the gradient of the objective function is given by

$$\begin{aligned}
\nabla_z f(z) &= \sum_{i=1}^n y_i \frac{\nabla_z \pi_i(z)}{\pi_i(z)} + (1 - y_i) \frac{-\nabla_z \pi_i(z)}{1 - \pi_i(z)} \\
&= \sum_{i=1}^n y_i \frac{\pi_i(z)(1 - \pi_i(z))\tilde{x}_i}{\pi_i(z)} - (1 - y_i) \frac{\pi_i(z)(1 - \pi_i(z))\tilde{x}_i}{1 - \pi_i(z)} \\
&= \sum_{i=1}^n y_i(1 - \pi_i(z))\tilde{x}_i - (1 - y_i)\pi_i(z)\tilde{x}_i \\
&= \sum_{i=1}^n (y_i - \pi_i(z))\tilde{x}_i
\end{aligned} \tag{7}$$

And thus, the Hessian is

$$\nabla_z^2 f(z) = \sum_{i=1}^n \pi_i(z)(\pi_i(z) - 1)\tilde{x}_i\tilde{x}_i^\top. \tag{8}$$

2. STEEPEST DESCENT WITH BACKTRACKING

Algorithm 1: General Line Search Method

```

1 input  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , starting point  $x_0 \in \mathbb{R}^n$ 
2 begin
3    $k := 1$ 
4   while converge criterion not satisfied
5     specify a descent direction  $d^k$  of  $f$  at  $x^k$ .
6     determine a step size  $\alpha_k > 0$  with  $f(x^k + \alpha_k d^k) < f(x^k)$ .
7     set  $x^{k+1} := x^k + \alpha_k d^k$ 
8     set  $k := k + 1$ 
9   end
10 end

```

The algorithm given in Algorithm 1 is a generalized line search. The steepest descent corresponds to choosing the direction with greatest change, i.e., the (normalized) gradient of the function. Therefore, we use $d_k = \nabla f(x_k) =: g_k$.

2.1. Step size.

2.1.1. Armijo Rule.

Let $\sigma \in (0, 1)$ be fixed. The *Armijo rule* is a condition on the step size which ensures a sufficient descent in the following sense:

$$f(x + \alpha d) \leq f(x) + \sigma \alpha \nabla f(x)^\top d \tag{9}$$

That is, we restrict the step size to one which ensures that the function at the next iteration is below the line $f(x) + \sigma \alpha \nabla f(x)^\top d$. Note that this line depends linearly on α , so if we take bigger steps, the function must be much smaller.

In general, one may modify the step size to approach the required condition in various ways. The approach we take is to simply multiply by a constant factor $\rho = 0.5$.

Algorithm 2: Backtracking

```

1 input descent direction  $d$ 
2 begin
3    $l := 0$ 
4    $\alpha^{(0)} := 1$ 
5   while Equation 9 not satisfied
6     determine  $\alpha^{(l+1)} = \alpha^{(l)} * \rho$ 
7     set  $l := l + 1$ 
8   end
9    $\alpha_k = \alpha^{(l)}$ 
10 end

```

ALGORITHM 2. Backtracking algorithm to determine a step size.

3. TRUST REGION METHODS

3.1. General idea.

Trust region methods are similar to line search algorithms, in that to find a step, one approximates the objective function. In each iteration, these methods define a *trust region*, in which we assume that our model will adjust well to the objective function.

Let $m_k(p)$ denote the model of the objective function. The step is calculated as an approximate minimizer of the model $m_k(p)$, restricted to the trust region $\mathcal{R}_c$.

$$p_k = \arg \min_p m_k(p), \text{ s.t. } p \in \mathcal{R}_c. \quad (10)$$

The descent direction and the step size are calculated simultaneously. This differs from the traditional line search method.

If the step obtained in the trusted region did not adjust *properly* to the function, then we reduce the size of our trust region. Then, a new step is calculated.

3.2. Previous Assumptions.

We will assume that the model m_k is quadratic at each x_k . By Taylor's approximation of second order,

$$f(x_k + p) = f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top \nabla^2 f(x_k + tp) p \quad (11)$$

with $t \in (0, 1)$. If we approximate the Hessian matrix, we have

$$m_k(p) = f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top B_k p \quad (12)$$

for some symmetric B_k .

We will also assume that the trust region is a circle, and its size will be completely determined by its radius.

3.3. Step search method.

To find an appropriate step, we solve the following restricted optimization problem:

$$p_k^* = \arg \min_p m_k(p) = f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top B_k p \quad (13)$$

such that $\|p\| \leq \Delta_k$, where Δ_k is the radius of the trust region. If moreover $B_k \succ 0$, the solution is given by

$$p^* = B_k^{-1} \nabla f(x_k). \quad (14)$$

If additionally $\|p^*\| \leq \Delta_k$, then the solution to the problem is precisely the one given above: $p_k^* = B_k^{-1} \nabla f(x_k)$.

To ensure that our trust region radius is *good* enough, we measure it with the following constant:

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)}. \quad (15)$$

Given that for all k we have $m_k(0) > m_k(p_k)$, the sign of the constant only relies on the numerator. That is, if $\rho_k < 0$, then $f(x_k) < f(x_k + p_k)$ and thus the function increases, so we dispose of this p_k . If $p_k \approx 1$, then the model approaches the behavior of the objective function, so we increment the radius of our trust region, in search for even better steps. If $0 \ll p_k < 1$ but close to 1, then Δ_k is not modified in the next iteration, so $\Delta_{k+1} = \Delta_k$. If $\rho_k > 0$ and close to 0, we reduce our trust region.

Algorithm 3: Trust Region

```

1 input  $\hat{\Delta} > 0, \Delta_0 \in (0, \hat{\Delta}), \eta \in [0, \frac{1}{4})$ 
2 begin
3   for  $k = 1, 2, \dots$ 
4     Compute  $p_k$  (minimize quadratic model in the trust region)
5     Compute  $\rho_k$ 
6     if  $\rho_k > \eta$ 
7        $| x_{k+1} = x_k + p_k$ 
8     else
9        $| x_{k+1} = x_k$ 
10    Compute  $\Delta_{k+1}$  given  $(\rho_k, \Delta_k, \widehat{\Delta}_k, \eta)$ 
11  end
12 end

```

Theorem 3.3.1. p^* is a global solution to the problem

$$\min_p m_k(p) = f(x_k) + g^\top p + \frac{1}{2} p^\top B p \quad (16)$$

such that $\|p\| \leq \Delta$ if and only if, p^* is feasible (i.e. the norm is bounded by Δ) and there exists $\lambda \geq 0$ such that

$$\begin{aligned} (B + \lambda I)p^* &= -g \\ \lambda(\|p^*\| - \Delta) &= 0 \\ B + \lambda I &\succeq 0 \end{aligned} \quad (17)$$

We note that if $\lambda = 0$ then $Bp^* = -g$. In particular, if $B \succ 0$, then $p^* = -B^{-1}g$ is the approximated Newton step. If $\lambda > 0$, then $Bp^* + \lambda p^* = -g$, so $\lambda p^* = -(Bp^* + g)$ and thus $p^* = -\frac{1}{\lambda} \nabla m_k(p^*)$. Therefore, p^* is parallel to $\nabla m_k(p^*)$, so p^* is the steepest descent direction.

3.4. Cauchy Point.

The *Cauchy Point* is the minimizer to the model m_k along the direction of steepest descent of the function f , that is, $-\nabla f(x_k)$ restricted to the trust region.

To find this step, we solve the following restricted optimization problem:

$$p_k^* = \arg \min_p m_k(p) = f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top B_k p, \quad (18)$$

such that $\|p\| \leq \Delta_k$. In practice, an approximation to this solution is enough. This point quantifies sufficient descent in a model. The Cauchy Point is denoted by p_k^C .

In an algorithm, we first need to solve the linear problem

$$p_k^S = \arg \min_p f(x_k) + \nabla f(x_k)^\top p \quad \text{s.t.} \quad \|p\| \leq \Delta_k \quad (19)$$

Then we find $\tau_k > 0$ that solves

$$\tau_k = \arg \min_{\tau \geq 0} m_k(\tau p_k^S), \quad \text{s.t.} \quad \|\tau p_k^S\| \leq \Delta_k \quad (20)$$

Then, we calculate the Cauchy point as $p_k^C = \tau_k p_k^S$. We describe this in the following algorithm:

Algorithm 4: Cauchy Point

```

1 input  $f(x_k), \nabla f(x_k), \Delta_k$ 
2 begin
3   Find  $p_k^S$  that solves Equation 19
4   Find  $\tau_k$  that solves Equation 20
5   return  $p_k^C = \tau_k p_k^S$ 
6 end
```

We may prove that this leads to

$$p_k^C = -\tau_k \frac{\Delta_k}{\|\nabla f(x_k)\|} \nabla f(x_k) \quad (21)$$

for

$$\tau_k = \begin{cases} 1 & \text{if } \nabla f_k^\top B_k \nabla f_k \leq 0 \\ \min\left(1, \frac{\|\nabla f_k\|^3}{\Delta_k \nabla f_k^\top B_k \nabla f_k}\right) & \text{otherwise} \end{cases} \quad (22)$$

3.5. Dogleg Method.

An augmented method that allows for better performance is the *dogleg* method. We combine the strengths of the Cauchy point and add in some optimizations for particular cases.

We assume that B_k is a positive definite matrix. Otherwise, we simply use Cauchy's point. The optimal point in a trust region $\Delta \in [0, \|p_k^B\|]$ the optimal trajectory is given in Figure 1.

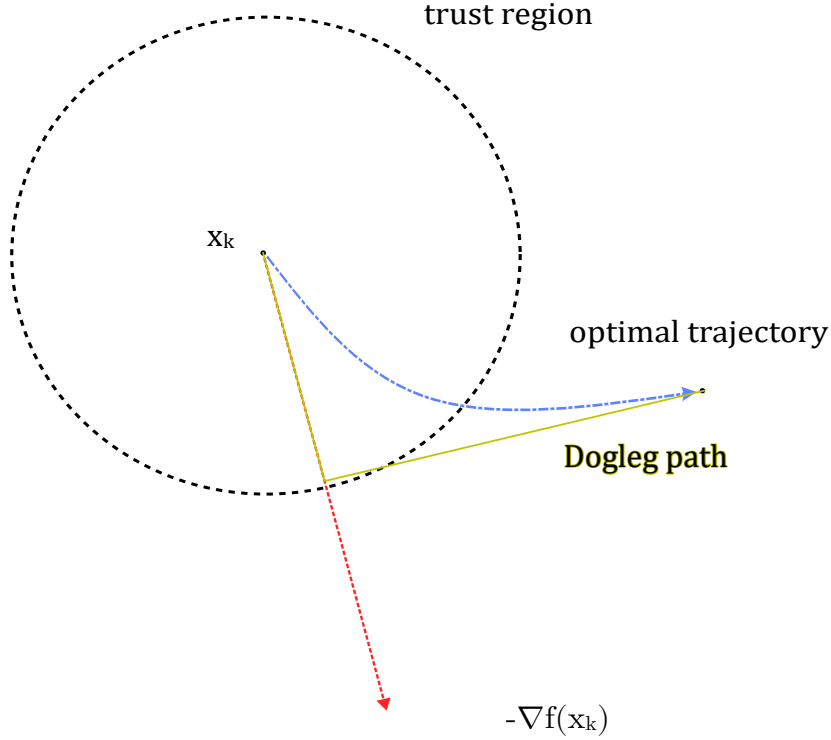


FIGURE 1. How a dogleg path is constructed.

The main idea of the dogleg method is to:

- minimize the model along the gradient $p_k^U = \alpha \nabla f_k$.
- minimize the unrestricted quadratic model if B_k is positive definite and obtain the complete step $p_k^B = -B_k^{-1} \nabla f_k$
- compute the step size of $p_k = F(p_k^U, p_k^B)$, a function that depends on both the complete step and the step of steepest descent.

The dogleg trajectory has the following parametrization:

$$\tilde{p}(\tau) = \begin{cases} \tau p_k^U & \text{if } 0 \leq \tau \leq 1 \\ p_k^U + (\tau - 1)(p_k^B - p_k^U) & \text{if } 1 \leq \tau \leq 2 \end{cases} \quad (23)$$

One then finds that

$$\begin{aligned} p_k^U &= -\frac{\nabla f_k^\top \nabla f_k}{\nabla f_k^\top B_k \nabla f_k} \nabla f_k \\ p_k^B &= -B_k^{-1} \nabla f_k \end{aligned} \quad (24)$$

So we need to find a minimizer τ^* for Equation 23.

Algorithm 5: Dogleg Trust Region

```

1 input  $\hat{\Delta} > 0$ ,  $\Delta_0 \in (0, \hat{\Delta})$  and  $\eta \in [0, \frac{1}{4}]$ 
2 begin
3   for  $k = 0, 1, 2, \dots$ 
4     begin
5       Compute  $p_k$  using Dogleg
6       Compute  $\rho_k$ 
7       if  $\rho_k > \eta$ 
8          $| x_{k+1} = x_k + p_k$ 
9       else
10         $| x_{k+1} = x_k$ 
11        Compute  $\Delta_{k+1}$  using  $(\rho_k, p_k, \Delta_k, \hat{\Delta}_k)$ 
12 end
```

4. IMPLEMENTATIONS

We use [1] to implement both algorithms. We provide some snippets below, but the full code can be found attached to this document.

4.1. Line Search with Backtracking.

```

for k in 1:max_iter
    gk = g(xk)

    if norm(gk) < tol
        println("Converged in $k iterations.")
        return xk, f(xk), k
    end

    pk = -gk
    αguess = min(1.0, 1.0 / norm(gk))

    αk = backtracking_armijo(f, gk, xk, pk, α0 = αguess)

    xk += αk * pk
end

```

LISTING 1. Snippet of steepest descent with Armijo backtracking.

4.2. Dogleg.

```

for k in 1:max_iter
    gk = g(xk)
    if (norm(gk) < tol)
        println("Converged in $k iterations.")
        return xk, f(xk), k
    end

    Bk = H(xk)
    pk = step_finder(gk, Bk, Δk)

    reduction = f(xk) - f(xk + pk)
    model_reduction = -(dot(gk, pk) + 0.5 * dot(pk, Bk * pk))

    ρk = reduction / model_reduction
    if ρk < 0.25
        Δk *= 0.25
    elseif (ρk >= 0.75 && isapprox(norm(pk), Δk, atol=1e-4))
        Δk = min(2*Δk, Δ)
    end

    if ρk > η
        xk += pk
    end
end

```

LISTING 2. Snippet of dogleg trust region method. We use the dogleg method as the `step_finder` function.

5. RESULTS

To run the experiments, use

```
julia main.jl
```


this will print the time taken by each method, its iterations, the plots will be saved as images, and the error is printed.

We run the methods with 1 iteration to compile the code (Julia is JIT compilation), so that the actual tests do not take the compilation time into account.

Note that we capped the iterations of the steepest descent to 5000, as it seemed pointless to wait for it to converge. The difference is clear.

The time taken for the steepest descent was 124.63 seconds (did not converge after 5000 iterations), while the time for the trust region took only 5.39 seconds.

Method	Time	f_k	$\ g_k\ $	Iterations
Gradient Descent	124.63	0.0318	0.0001	5000
Dogleg	5.39	0.0318	7.86e-7	23

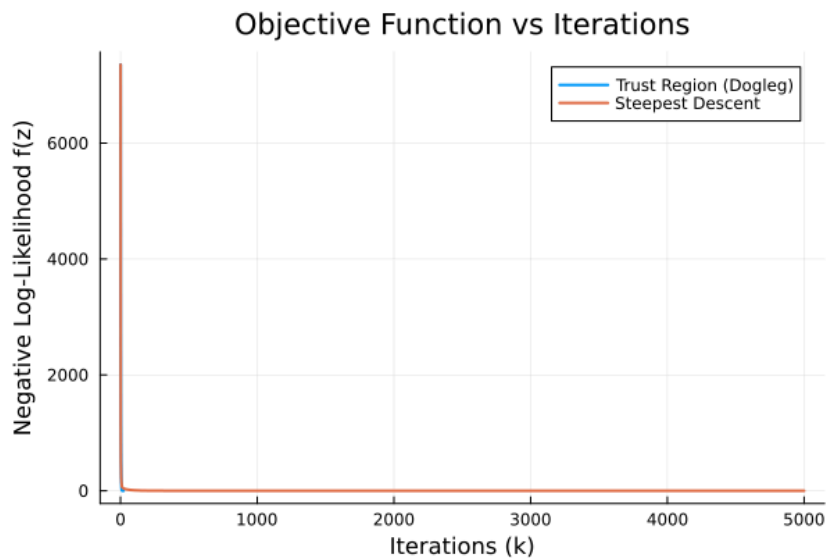


FIGURE 2. Iterations vs $f(x_k)$. Both methods arrive to zero rapidly. However, the steepest descent method does not converge rapidly. See the next graph.

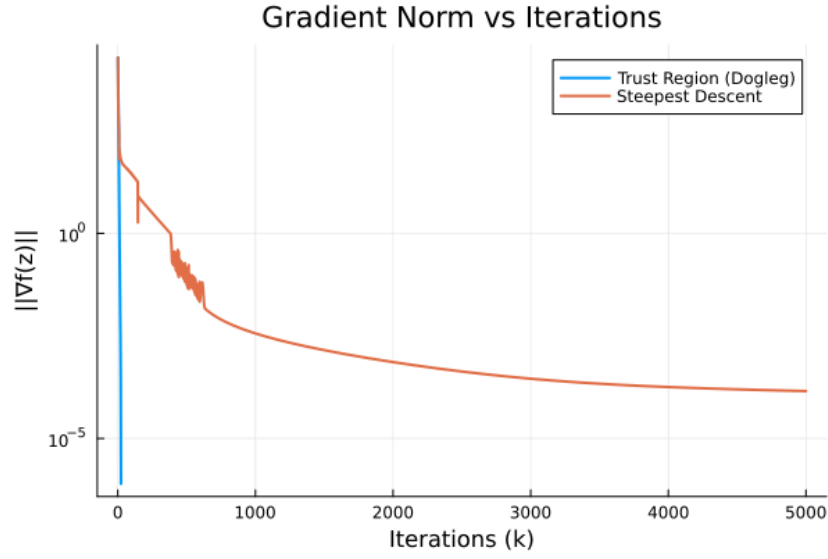


FIGURE 3. Iterations vs gradient. The trust region converges rapidly, while gradient descent follows an exponential decrease. The log scale makes this look nicer.

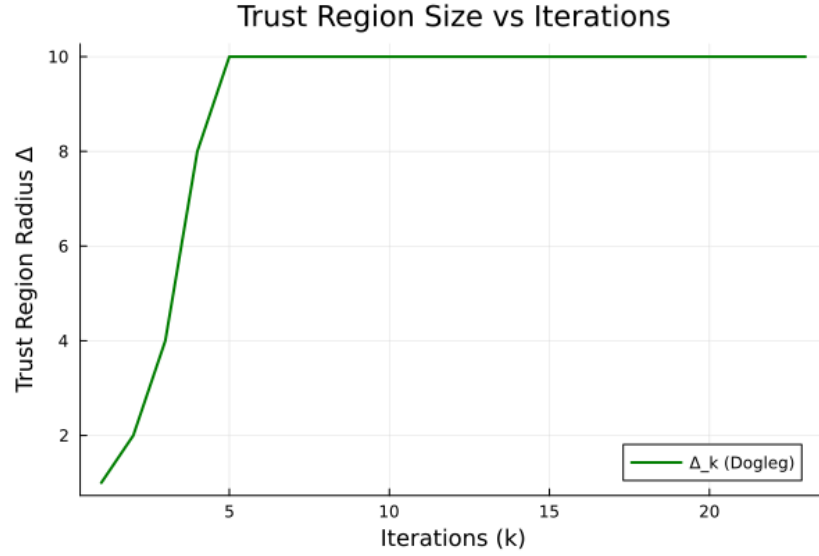


FIGURE 4. Iterations vs Δ_k . The region stabilizes to the maximum set region size after 5 iterations.

5.1. Classification Error.

We define this as

$$\text{error} = \frac{1}{n} \sum_{i=1}^n |\mathbf{1}_{\pi_i(\hat{z}) > 0.5} - y_i| \quad (25)$$

In both cases, we obtain an error of 0.05%. Note that minimizing the objective function and minimizing the error function are different problems, we cannot apply the methods we described earlier as the error function is not differentiable. However, minimizing the error function is more practical from a real-world viewpoint.

6. CONCLUSIONS

The trust region method allows for a faster convergence on the MNIST dataset. Although each iteration is more expensive, the number of iterations needed to converge is significantly lower than steepest descent. Both methods obtain an error in classification of 0.05%.

REFERENCES

1. Bezanson, J., Edelman, A., Karpinski, S., Shah, V.B.: Julia: A fresh approach to numerical computing. *SIAM Review*. 59, 65–98 (2017). <https://doi.org/10.1137/141000671>

Email address: pedro.llerenas@cimat.mx